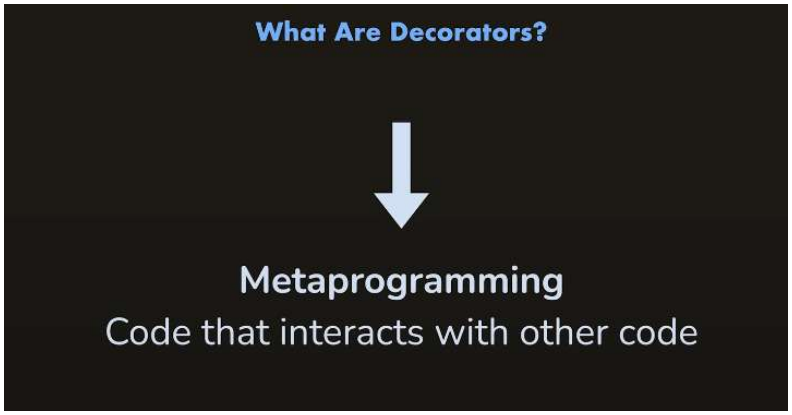
Section 11: ECMAScript Decorators

2026年6月8日 9:22

What Are Decorators? And ECMAScript Decorators vs Experimental Decorators



So what are decorators and what's the idea behind them? Well, decorators, not just in TypeScript, but in programming in general, because this feature exists in other programming languages as well, are a metaprogramming feature.

And metaprogramming is simply code that you write that interacts with other code, and that other code then makes up the actual application,

And typically decorators look something like this. For example, this is how they are used in TypeScript. They start with an @ symbol or you have to add such an @ symbol in front of the decorator name. They may take arguments, but they don't have to. And they can then be attached to things, so for example, to a property of a class here.

that the decorator that's attached to a thing changes that thing. For example, here, this length decorator is coming from a third party library called Class Validator, and it adds validation logic to the property

to which it is added, ensuring that this title in this class has a certain min and max length.

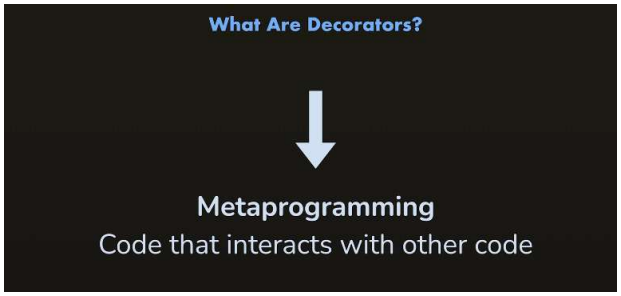
But you can, of course, implement any kind of logic with decorators.

The core idea, however, and that's important, is that decorators are a feature that allow you to write code

that changes the behavior of other code. Like here, where a decorator changes the behavior of the title property.

The end. The idea behind decorators is that you write code in a certain way
so that you can alter the behavior of some other code to which you attach these decorators.

That's the idea.

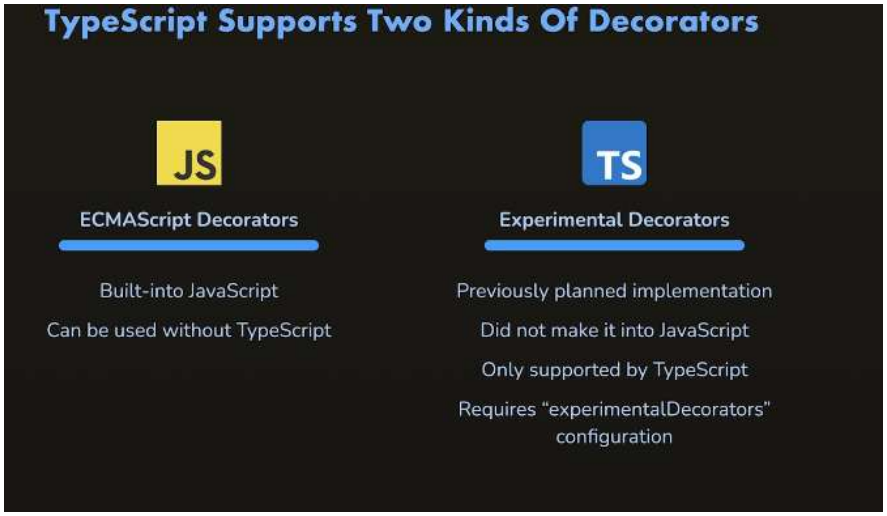


There are two different approaches or ways of writing decorators,
of building them in TypeScript.

Because in TypeScript, you can build both ECMAScript
and experimental decorators.

And both kinds of decorators can be built with TypeScript.
The difference is that the ECMAScript decorators,
the official decorators, if you want to call 'em like this,

are actually also a JavaScript feature.
So you could build them without TypeScript as well.
You don't need TypeScript to build ECMAScript decorators.



Currently when I'm recording this, it is a stage three proposal, which in the end means
that the proposal has been recommended for implementation, so that browser engines and all the other engines
that in the end execute JavaScript code should support that feature, you could say

or should support it in the future And once that's the case you'll be able to build such decorators
with just JavaScript without TypeScript.
But TypeScript also supports this feature,
and it does so already actually.
So even before all the different engines
implement this feature,

and for the time being,
also depending on your compilation target
set in the TS config file,
TypeScript will simply compile this decorator code
to code that will work in older engines,
other browsers and so on as well.

So you can use this feature today with TypeScript
at some point in the future.
You can also theoretically use it without TypeScript.
And then there are these experimental decorators,
and these decorators are built with a different syntax
and they're only available in TypeScript.

These decorators simply reflect
an older version of that implementation proposal
for decorators.
So a version that did not make it
to stage three, you could say,
a version that's not going to be implemented
in plain Vanilla JavaScript.

But TypeScript started supporting this old proposal
many, many years ago already,
and it still does and will for the foreseeable future.
But this proposal did not make it into JavaScript,
that's why it's only available in TypeScript.

And that's why in TypeScript,
you can build both kinds of decorators
with their difference syntaxes
because TypeScript does support both.
However, if you do want to build
such experimental decorators,
you must tweak your TS config,
and you must make sure that the experimental decorators flag
is set to true there.

That's important.
But we'll cover experimental decorators,
so this older approach, as you could say,
in a separate section in this course,
and it is definitely still a feature that's worth learning
because all existing projects
that use decorators in TypeScript
will be using that old approach.

But again, we do cover that in a separate section.
In this section here,
I'll get you started with the EXMAScript decorators,
and I'll introduce you to them,
show you how you can build decorators
and how you can use them

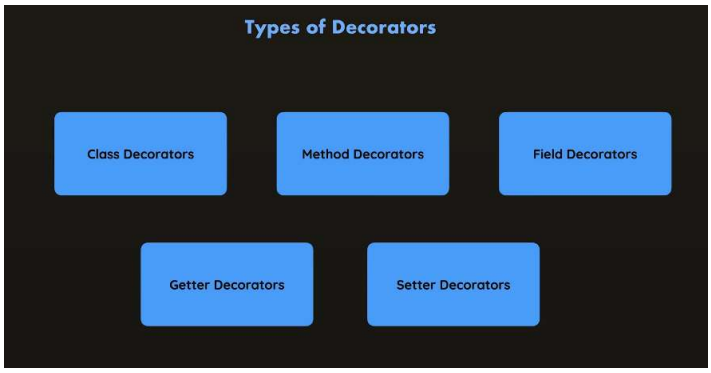
Exploring Different Types of Decorators

Now, before we get started
writing decorator code,

I also briefly want to introduce you
to the different types of decorators.
So not the different kinds,

the two different syntaxes you can use,
but different types of decorators.
And here it's important to understand
that this decorator's feature
is an object-oriented programming feature,

which means this decorator's feature,
the decorators you are going to build,
can only be used in conjunction with classes, their methods,
their fields, or getters and setters in the end.



Building a First Decorator

Decorator is an object oriented related feature, I will start by adding a class

that should be attached to the entire class

to interact with it, to interact with that code,

if you wanna call it like this.

And for that, we create a function

because decorators in JavaScript

are just functions.

but it is a function written in a certain way

receiving a certain amount of arguments and so on.

But it is just a function in the end.

And I'll name my function logger here,

but the name is up to you.

```
function logger() {}

class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

because I plan on creating a decorator that can be attached

to this class here

to log information about it, just to get started.

So we can try attaching this function

as a decorator to this class.

And how do we do that?

Well, by adding something in front of this class,

```
function logger() {}

@
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

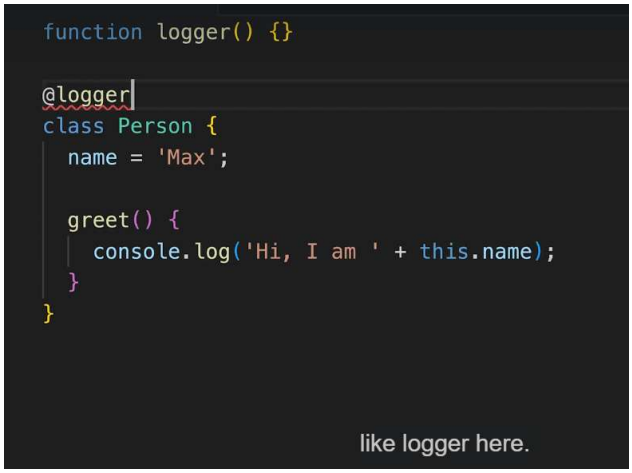
by adding the @ symbol.

That's an important symbol.

to a class, a field, a method, or a getter or a setter.

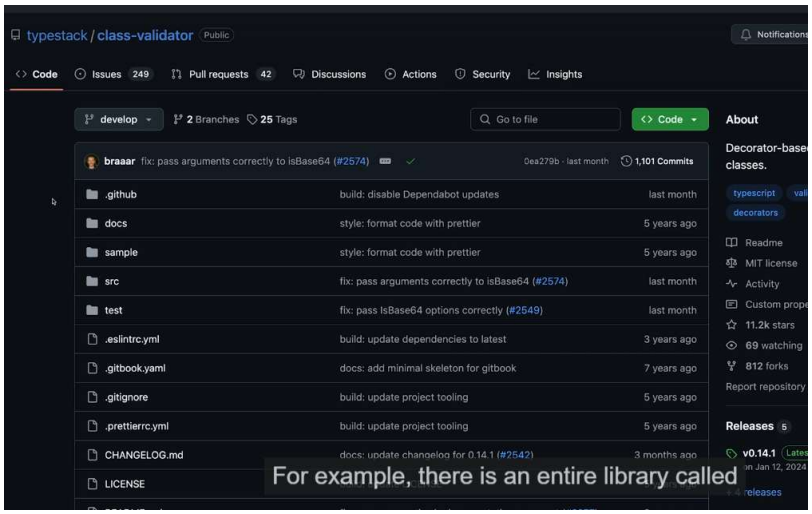
After @ symbol, you put the name of the function

you want to use as a decorator,

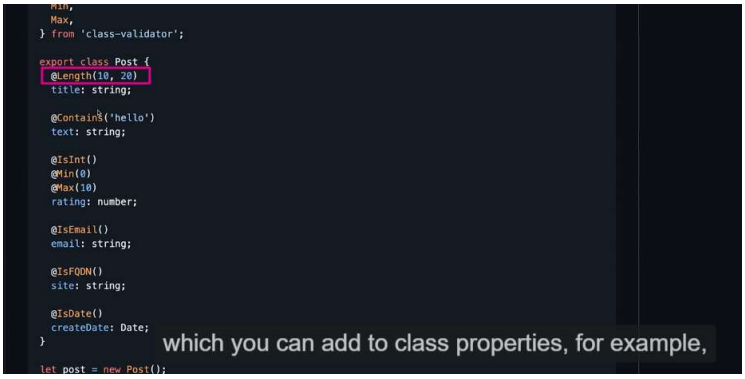


So it's code interacting with our code.

It's the logger function here, in this case,

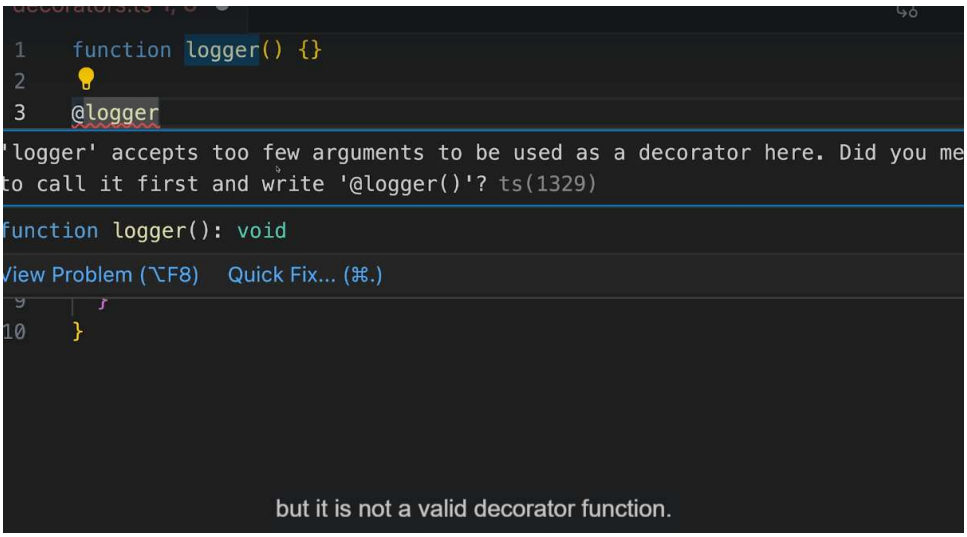


class validator, which you could install

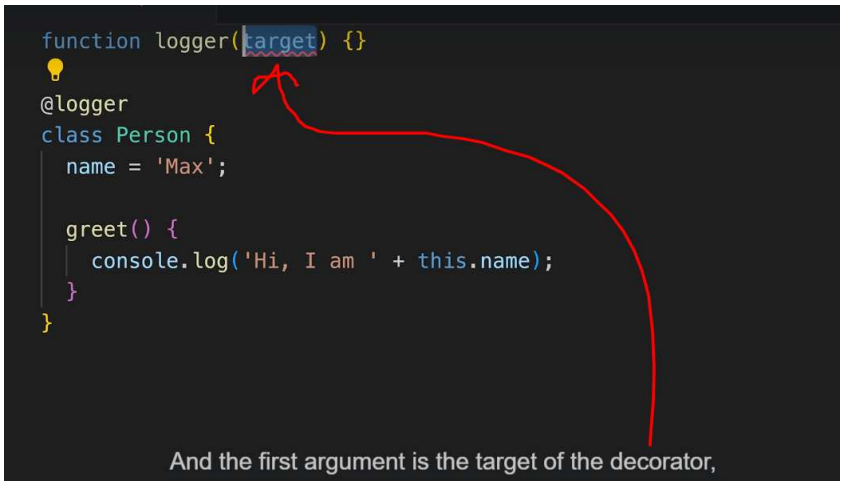


to add validation to those properties

You can use this feature to attach logic to your code.



For an ECMAScript decorator, it's two arguments.


function logger(target) {}
@logger
class Person {
name = 'Max';
greet() {
console.log('Hi, I am ' + this.name);
}
}
And the first argument is the target of the decorator,


function logger(target) {}
@logger
class Person {
name = 'Max';
greet() {
console.log('Hi, I am ' + this.name);
}
}
the thing you're attaching it to.


@logger
class Person {
name = 'Max';
greet() {
console.log('Hi, I am ' + this.name);
}
}
So in this case, this class.

but actually you typically accept two arguments

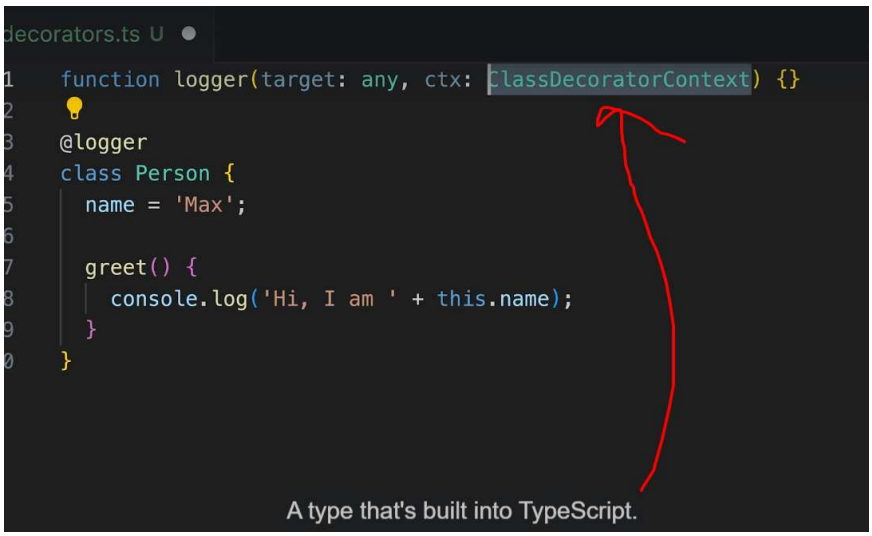
because the second argument is a context object

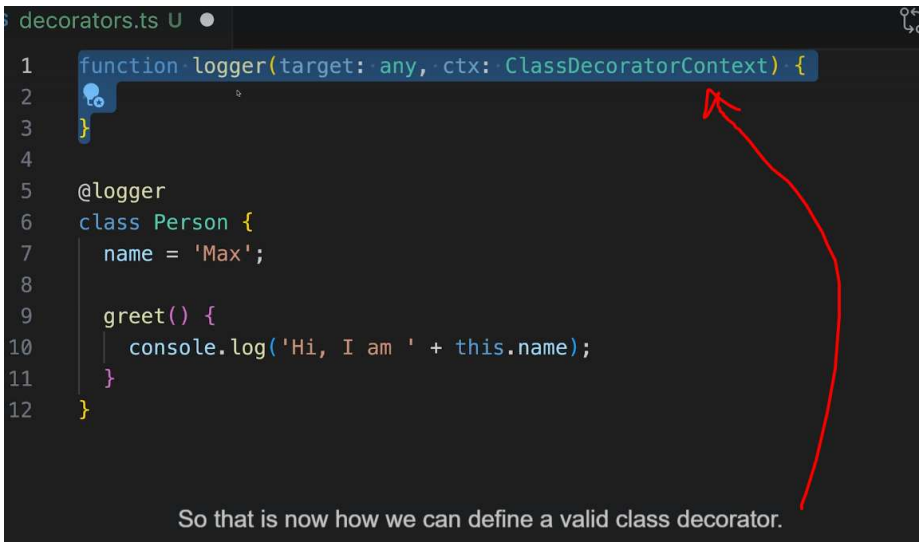
that gives you extra information about the thing

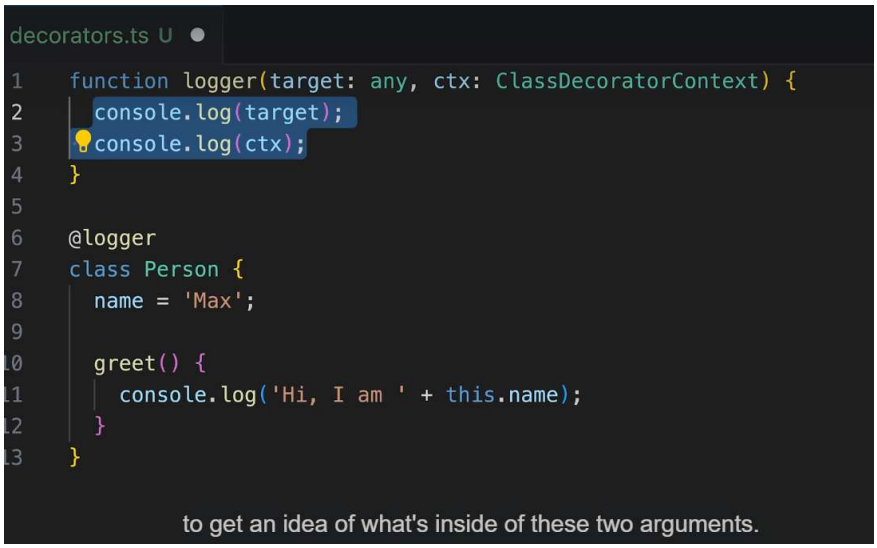
you're attaching the decorator to.

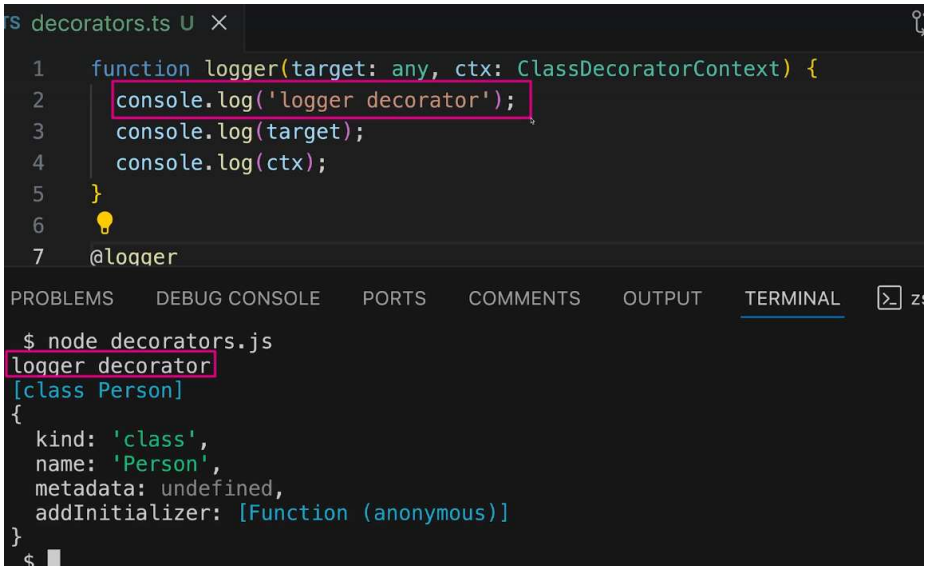
The context is an object with information about that thing.

And that context object is of type class decorator context.

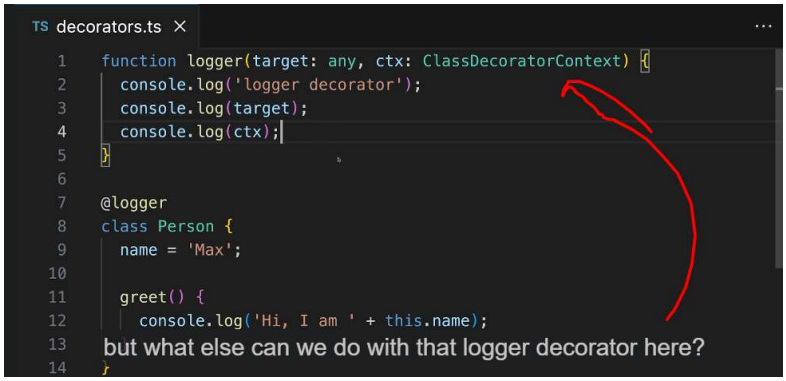








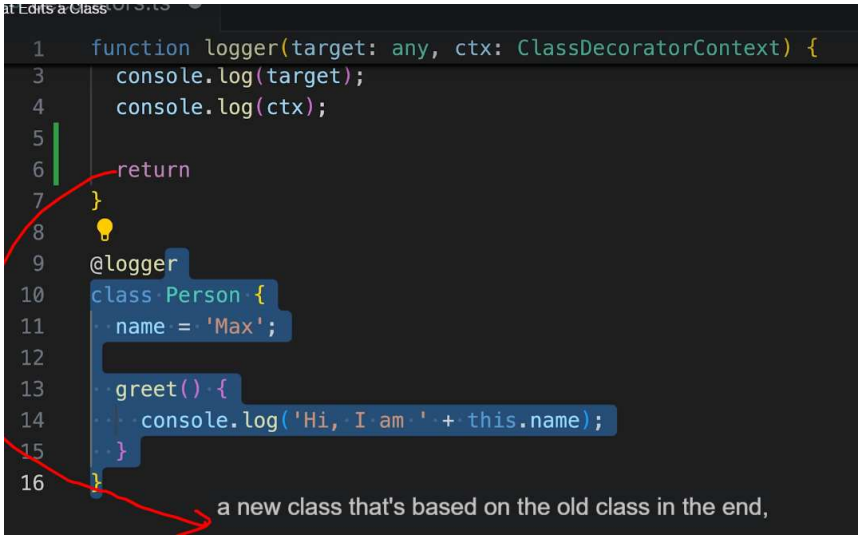
Building a Class Decorators That Edits a Class



Well, you can also use decorators

to change the thing you are attaching them to.

So to change the class, for example,



a new class that's based on the old class in the end,

and you can do that with a syntax that might look strange,

but that is valid JavaScript code.

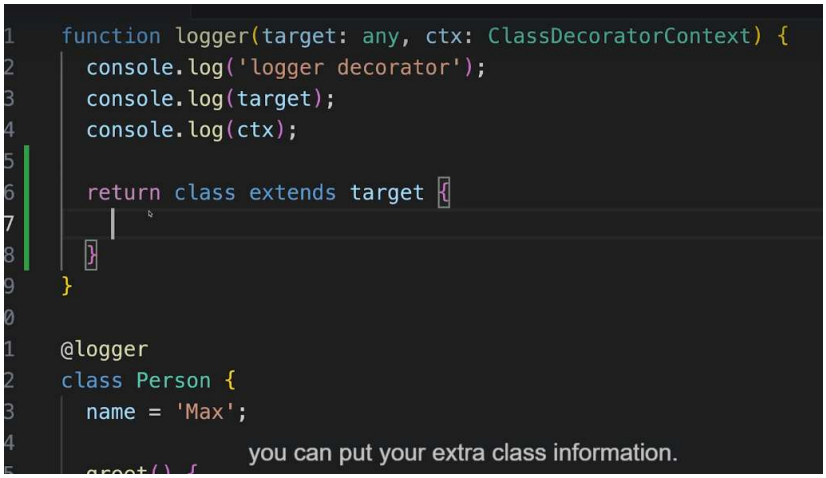
You can return an anonymous class

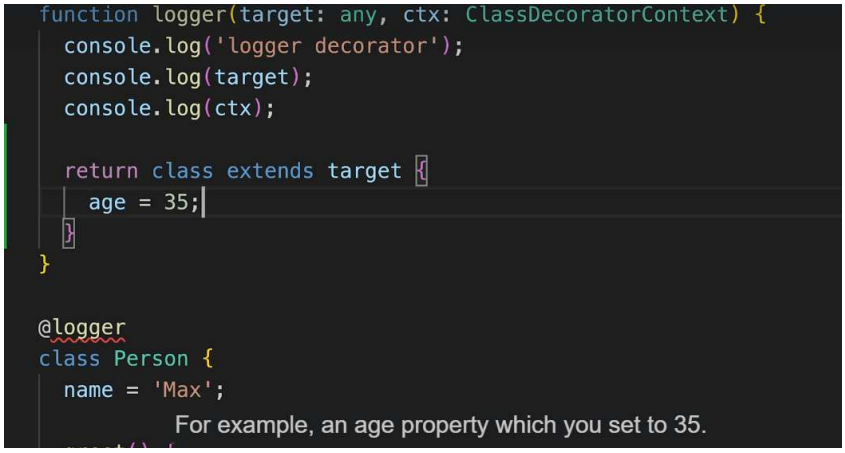
and then "extends target."

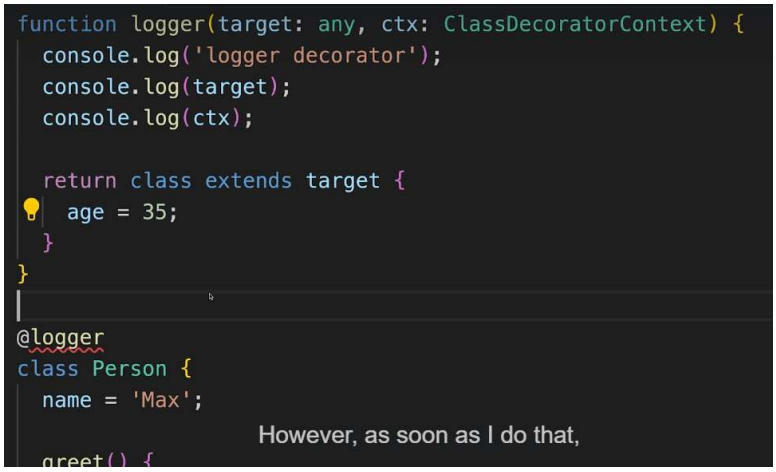
and I'm not giving my class a name here

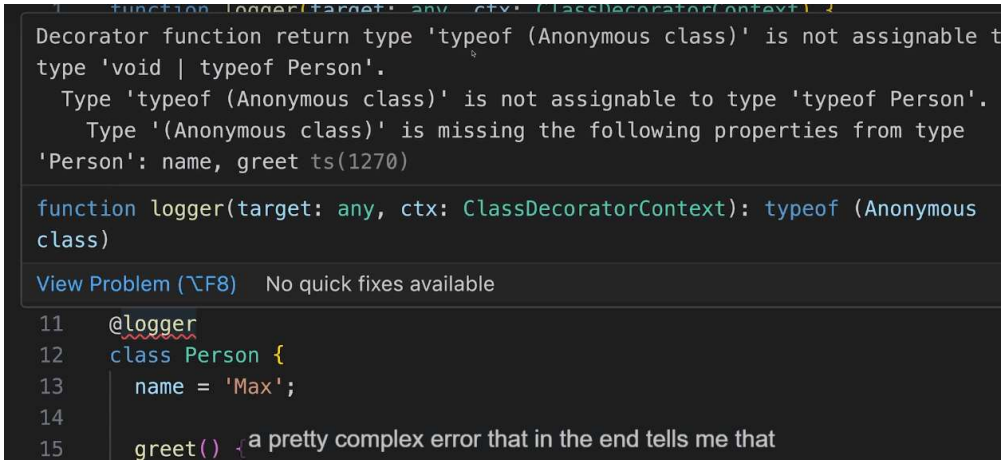
because it's an anonymous class,

which I'm just creating temporarily to return it,

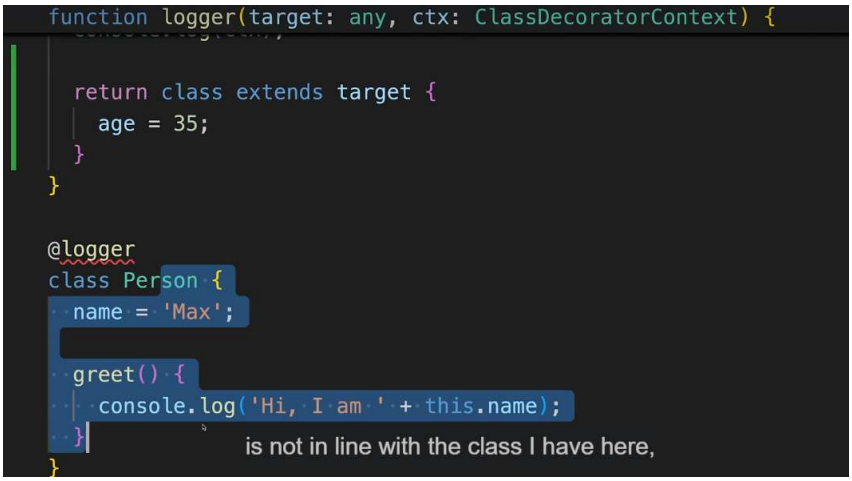

function logger(target: any, ctx: ClassDecoratorContext) {
console.log('logger decorator');
console.log(target);
console.log(ctx);
return class extends target {
}
}
@logger
class Person {
name = 'Max';
you can put your extra class information.

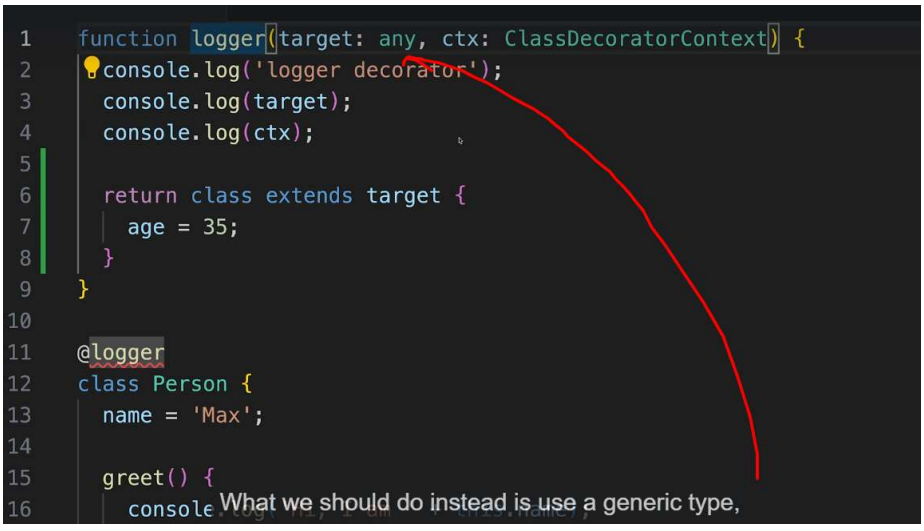

function logger(target: any, ctx: ClassDecoratorContext) {
console.log('logger decorator');
console.log(target);
console.log(ctx);
return class extends target {
age = 35;
}
}
@logger
class Person {
name = 'Max';
For example, an age property which you set to 35.

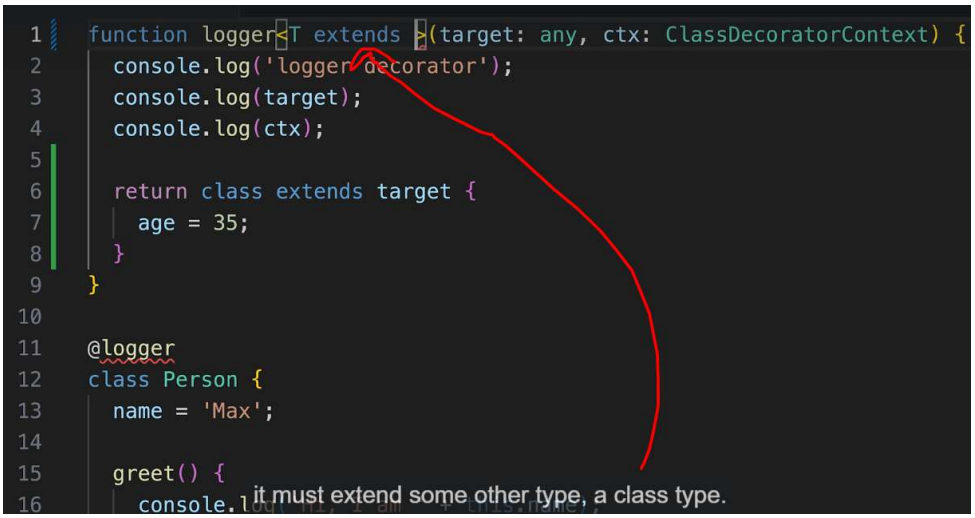

function logger(target: any, ctx: ClassDecoratorContext) {
console.log('logger decorator');
console.log(target);
console.log(ctx);
return class extends target {
age = 35;
}
}
@logger
class Person {
name = 'Max';
However, as soon as I do that,

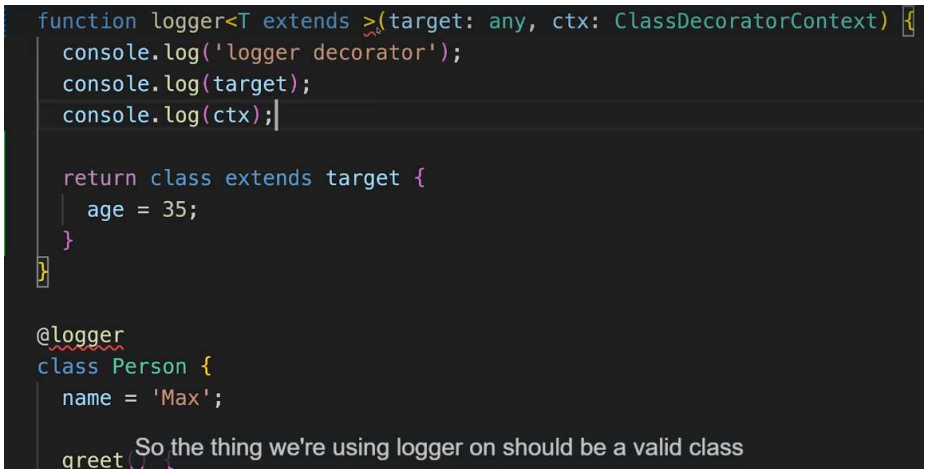


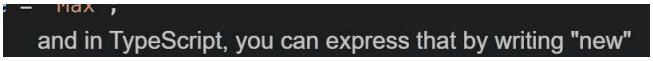
the kind of class I'm returning



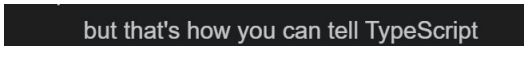



function logger<T extends >(target: any, ctx: ClassDecoratorContext) {
console.log('logger decorator');
console.log(target);
console.log(ctx);
return class extends target {
age = 35;
}
}
@logger
class Person {
name = 'Max';
greet() {
it must extend some other type, a class type.

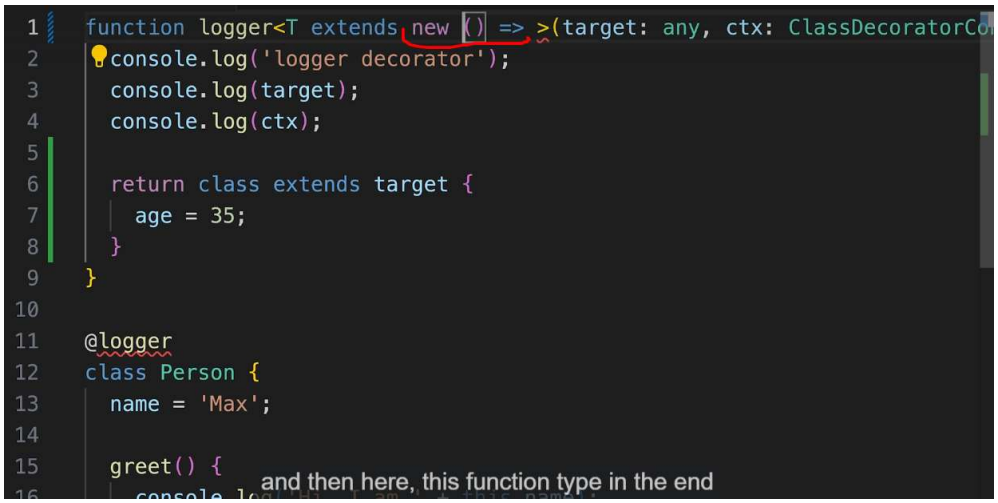

function logger<T extends >(target: any, ctx: ClassDecoratorContext) {
console.log('logger decorator');
console.log(target);
console.log(ctx);
return class extends target {
age = 35;
}
}
@logger
class Person {
name = 'Max';
So the thing we're using logger on should be a valid class


and in TypeScript, you can express that by writing "new"


function logger<T extends new () => >(target: any, ctx: ClassDecorator
console.log('logger decorator');
console.log(target);
console.log(ctx);
return class extends target {
age = 35;
}
}
@logger
class Person {
name = 'Max';
and then a function type thereafter.


but that's how you can tell TypeScript

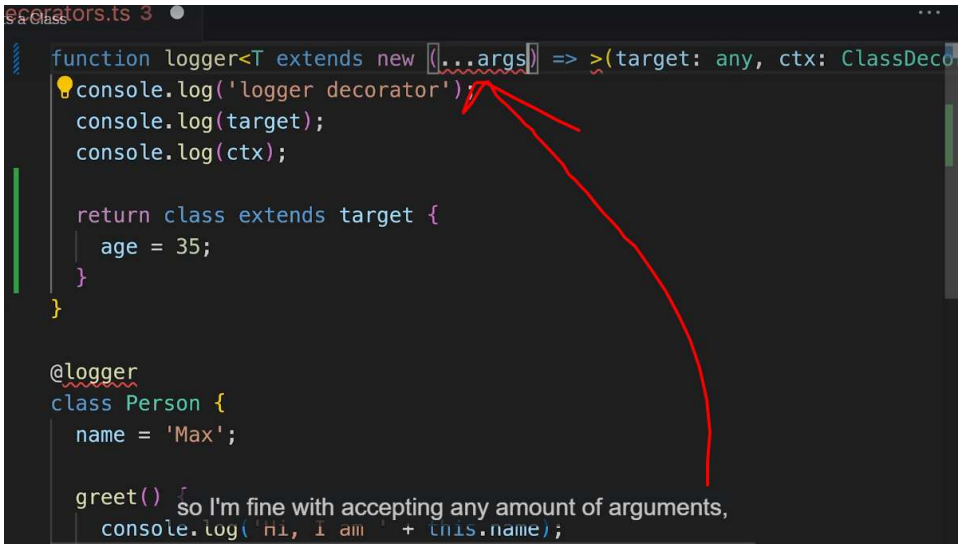
that you want to interact with a class



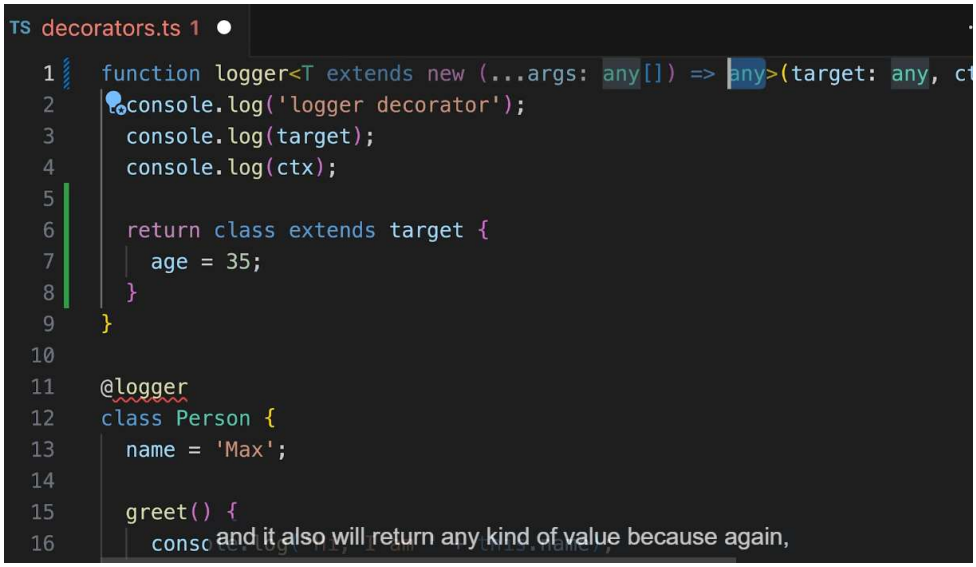
represents the class constructor

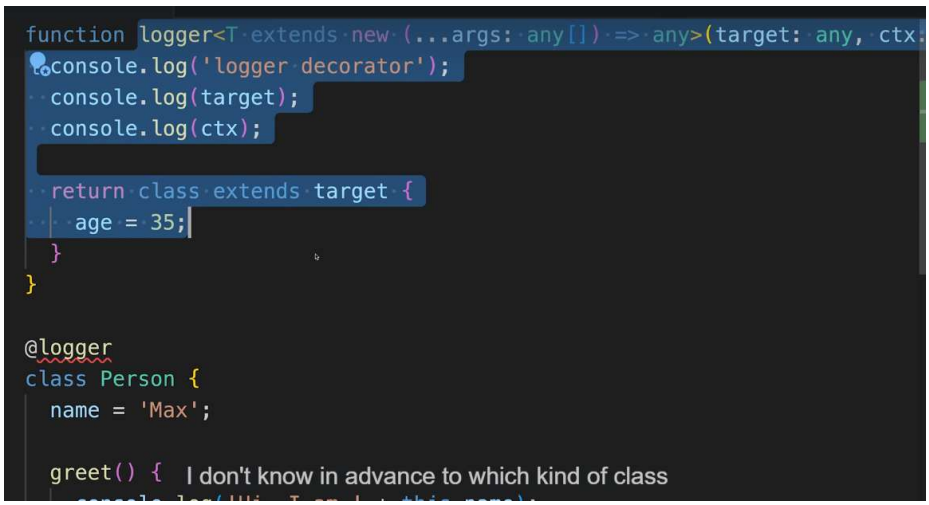
and there, I want to be able to use my logger decorator

on any kind of class,



That's what this syntax means.





x';

I want to attach this logger decorator.

```
function logger<T extends new (...args: any[]) => any>(target: any, ctx:
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';

  greet() {
```

So that's how you can express

() {

that you wanna base your type "T" on some class

() that accepts any kind of arguments in its constructor

where you then return any kind of value

and now it's this "T" type, this generic type,

```
function logger<T extends new (...args: any[]) => any>(target: T, ctx:
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';

  greet() {
```

which I use for target.

```ts
function logger<T extends new (...args: any[]) => any>(target: T, ctx:
  console.log('logger decorator');
  console.log(target);
  console.log(ctx);

  return class extends target {
    age = 35;
  }
}

@logger
class Person {
  name = 'Max';

  greet() {
```

this arrow down here goes away.

```ts
function logger<T extends new (...args: any[]) => any>(target: T, ctx: ClassDecoratorContext)
    return class extends target{
        age = 30;
    }
}

@logger
class Person{
    name = 'Khalid';

    greet(){
        console.log('Hi, I am '+ this.name);
    }
}
                + constructor Person(): Person
const person = new Person();
console.log(person);

//in the terminal
// tsc
```

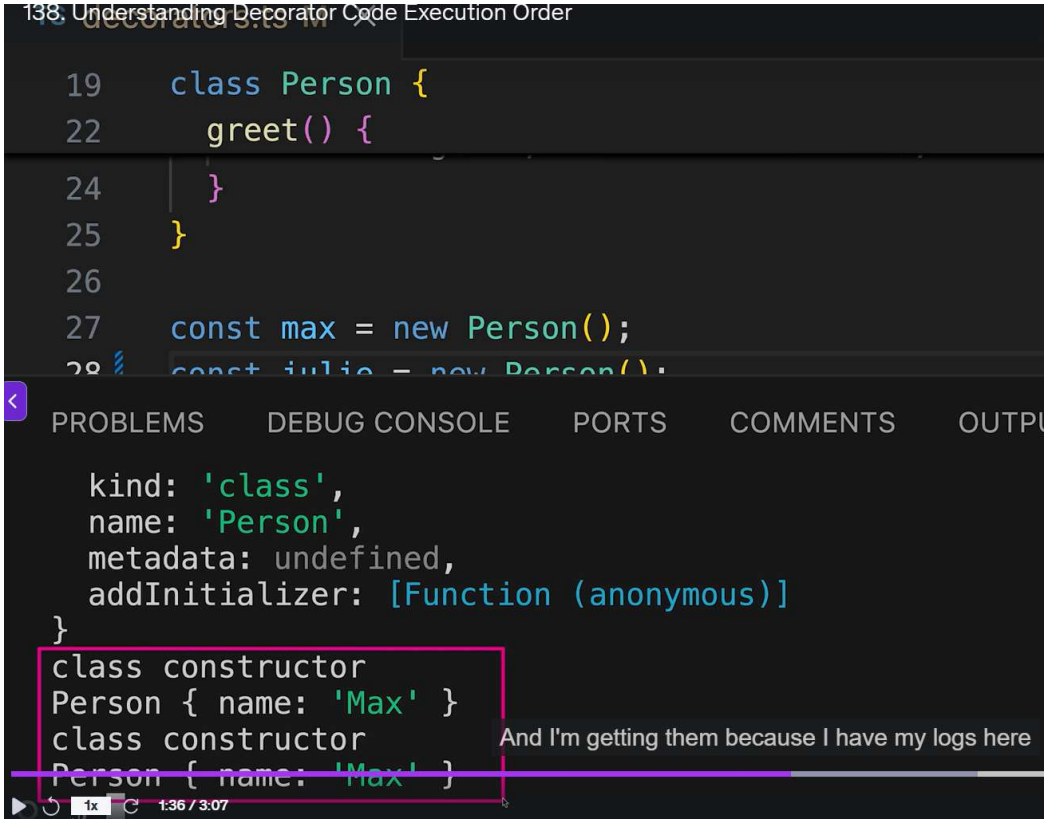
```ts
hat Edits a Class
  class Person {
    greet() {
      console.log('Hi, I am ' + this.name)
    }
  }

  const max = new Person();
```

```
PROBLEMS  DEBUG CONSOLE  PORTS  COMMENTS

logger decorator
[class Person]
{
  kind: 'class',
  name: 'Person',
  metadata: undefined,
  addInitializer: [Function (anonymous)]
}
Person { name: 'Max', age: 35 }
```

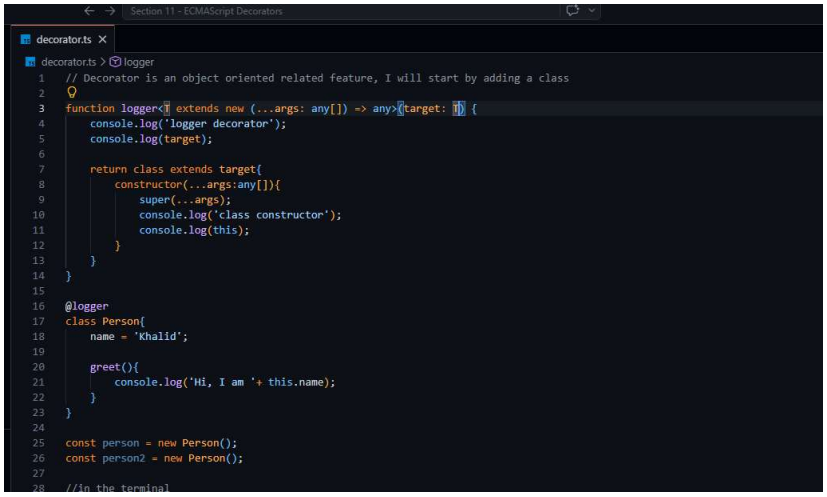
```
    age = 35;
  };
}

@logger
class Person {
  name = 'Max';

  greet() {
    console.log('Hi, I am ' + this.name);
  }
}
```

the original class does not get replaced by that class here,

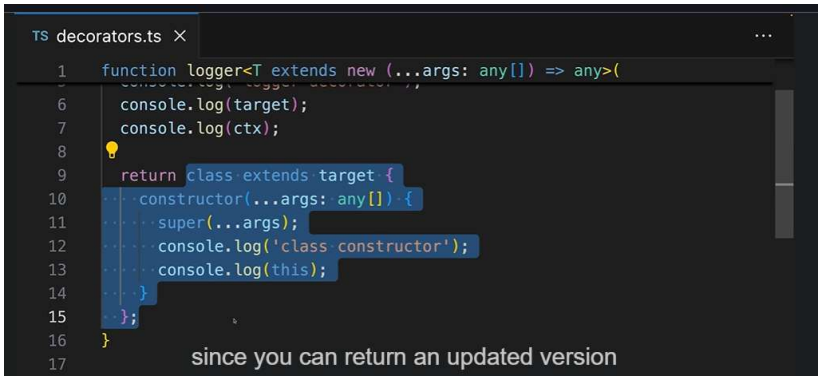
{ but in the end, we merged them, you could say.



Creating a Method Decorator

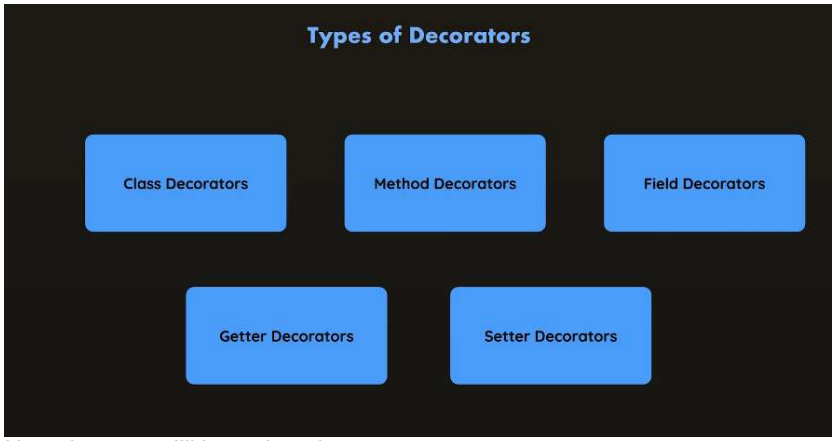


So that's how you can build
and use a decorator
and you can do anything you want with it in the end.



since you can return an updated version
of the thing you're attaching it to.
You can use it to add more properties, methods,
and that can make it a really powerful feature.
You could, for example, create a decorator
and share it as a library.

so that other developers can use it
that enhances any kind of class it's added to
by adding extra methods to it or anything like that.
So that's really a powerful feature to consider
and be aware of.

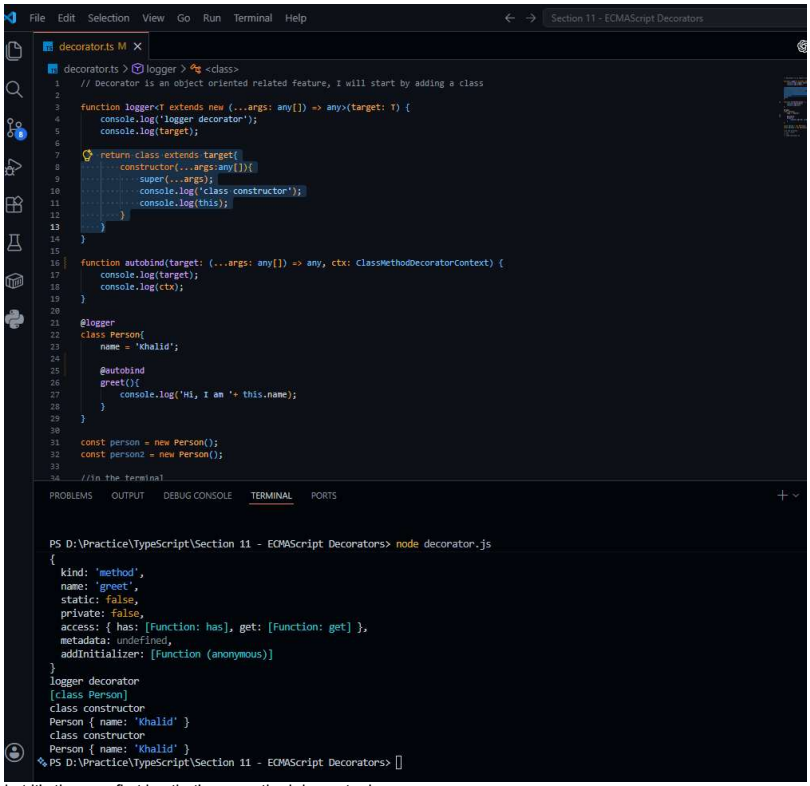


Here, however, I'll leave it at that
and instead move on to a different kind
of decorator you can build
because as mentioned,
you don't just have class decorators,
instead you also have decorators
that can be added to methods or fields.

So let's build a method decorator here,

```ts
function logger<T extends new (...args: any[]) => any>(
}

function autobind(
  target: (...args: any[]) => any,
  ctx: ClassMethodDecoratorContext
) {}

@logger
class Person {
  name = 'Max';

  @autobind
  greet() {
    console.log('Hi, I am ' + this.name);
```

Creating a Method Decorator



but it's the very first log that's my method decorator here
because the method gets initialized
before the class initialization is done.

That's why the decorator added to that method executes
before the class decorator executes
because all these decorators only execute once the thing
they have been attached to is done initializing.
And you see here, it's the greet function,

the greet method that's being logged,

Using Decorators to Solve A Common Problem

```
27  class Person {
30    @autobind
31    greet() {
32      console.log('Hi, I am ' + this.name);
33    }
34  }
35
36  const max = new Person();
37  const greet = max.greet;
38  greet();
```

If I run and compile this code

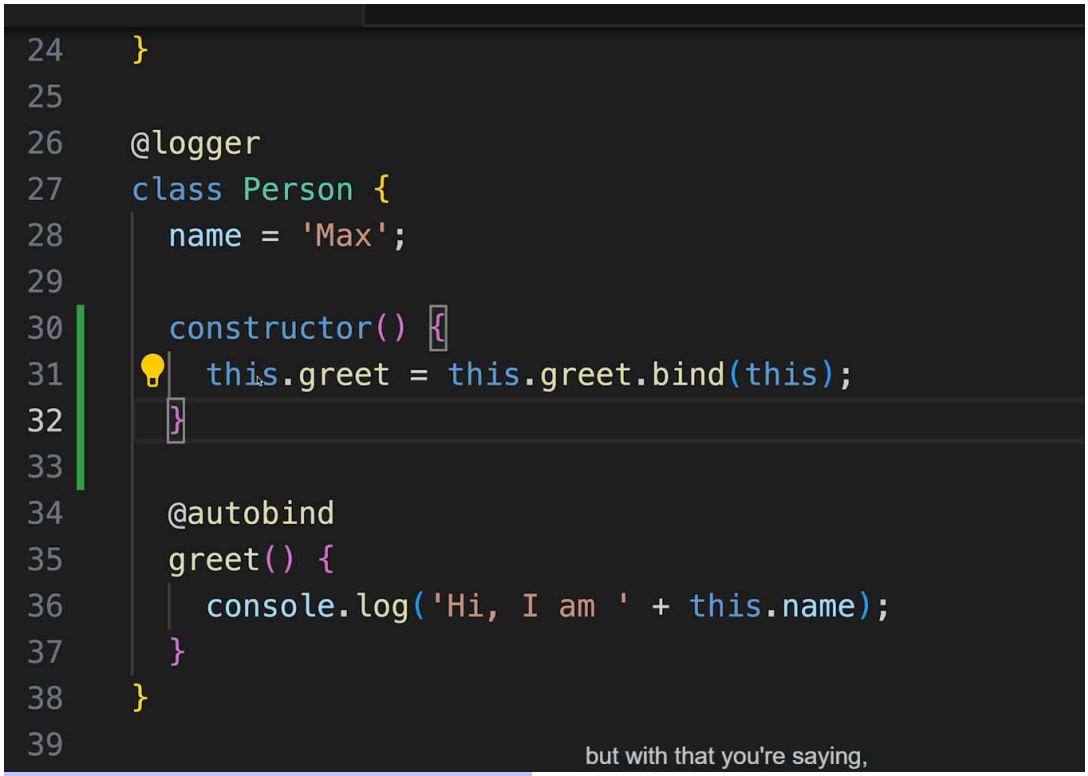
```
$ tsc
$ node decorators.js
```

I am getting an error here

```
PROBLEMS  DEBUG CONSOLE  PORTS  COMMENTS  OUTPUT  TERMINAL

            console.log('Hi, I am ' + this.name);
                                           ^

TypeError: Cannot read properties of undefined (reading 'name')
    at greet (/Users/max/development/teaching/understanding-typescri
tors/decorators.js:72:44)
    at Object.<anonymous> (/Users/max/development/teaching/understan
script/decorators/decorators.js:79:1)
```

What's the problem here? The problem is related to 'this' keyword in JavaScript. It's not typescript specific at all.

You can solve this problem by binding this class.

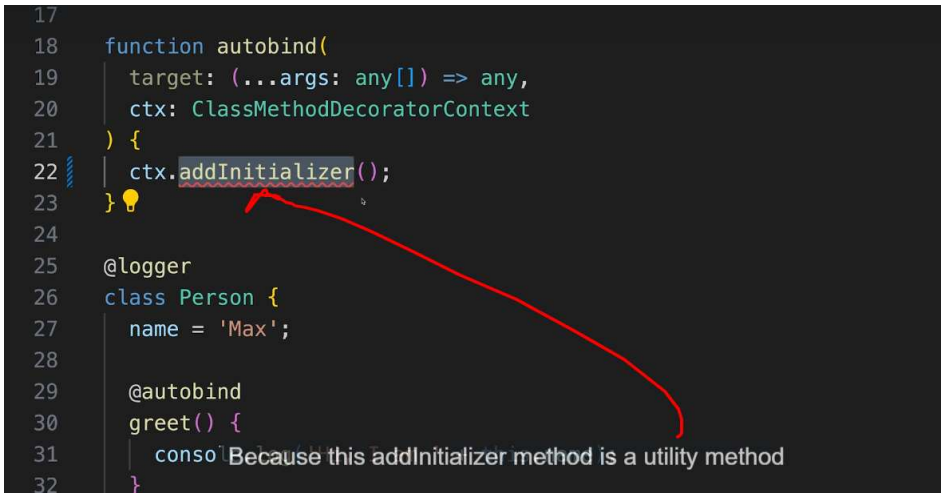


JS has a solution for that but it is kinda annoying.
Because in that case, you have to do it for every function in the constructor.
That's when decorator can solve this problem

Implementing A Decorator-based Solution: autobind

let's solve it with this decorator.
Because my idea is to put that code
which I had in the constructor here in this person class
in the last lecture into this decorator.

So that this decorator will automatically bind the method
it's attached to, to the class the method belongs to.



.log('Hi, I am ' + this.name);
to allow you to run code related to this thing

you are attaching the decorator to

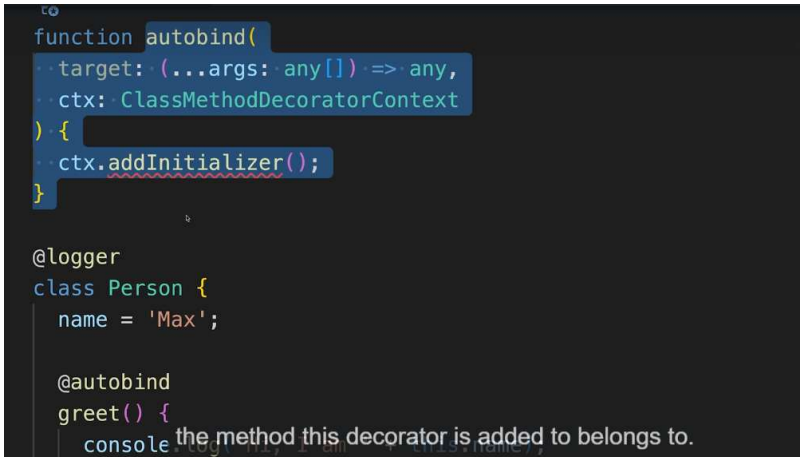
after the surrounding thing,

so the surrounding class is done initializing.

So in the end you can think

of this addInitializer method here as a way

of giving you access to the constructor of the class



```
function autobind(
  target: (...args: any[]) => any,
  ctx: ClassMethodDecoratorContext
) {
  ctx.addInitializer(function() {});
}

@logger
class Person {
  name = 'Max';

  @autobind
  greet() {
    console
```

For that you must pass a function to addInitializer

Introducing Field Decorator

So this was a method, decorator.
The last kind of decorator I wanna explore together
with you is the field decorator,

so a decorator that can be added to fields like name here.
And for that, I'll create another new function
because a decorator is still a function
and I'll simply name it fieldLogger
because I wanna keep this decorator simple
and just log something again.
Now, like before, we add it with the @ symbol,
and like before,

we must accept the appropriate amount of arguments.
So for that here, we need a target and a context.
And the type of target here
for fields will actually always be undefined
because the decorator code will be executed
before the field is done initializing.
That's important.

That's why the target, the value of the field,
will be undefined.
The value of the greet method was the function,
the actual method function.
The value of the class
for the class decorator was the finished class.
But the value, the target of the field decorator,
fieldLogger here will be before it's been initialized
and it will therefore always be undefined.

That's just something to keep in mind.
When creating an ECMAScript field decorator,
target will be undefined.
The context, however, will not be undefined.
Instead that will be a ClassFieldDecoratorContext object.

```
//Field Decorator
function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext){
    console.log(target);
    console.log(ctx);
}



@logger
class Person{
    @fieldLogger

    name = 'Khalid';

    @autobind
    greet(){
        console.log('Hi, I am '+ this.name);
    }
}
```

```
PROBLEMS  DEBUG CONSOLE  PORTS  COMMENTS  OUTPUT

$ tsc
$ node decorators.js
```

```
$ node decorators.js
undefined
{
  kind: 'field',
  name: 'name',
  static: false,
  private: false,
  access: { has: [Function: has], get: [Function: get], set: [Function: set]
}
```

undefined is logged, as I explained.

```
31
32  function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
33    console.log(target);
34    console.log(ctx);
35  }
36
37  @logger
38  class Person {
39    @fieldLogger
40    name = 'Max';
41
42    @autobind
43    greet() {
```

Now, just as with class and method decorators,

```
function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
  console.log(target);
  console.log(ctx);

  return
}

@logger
class Person {
  @fieldLogger
  name = 'Max';
```

you can also return something here

to change the thing you're attaching the decorator to.

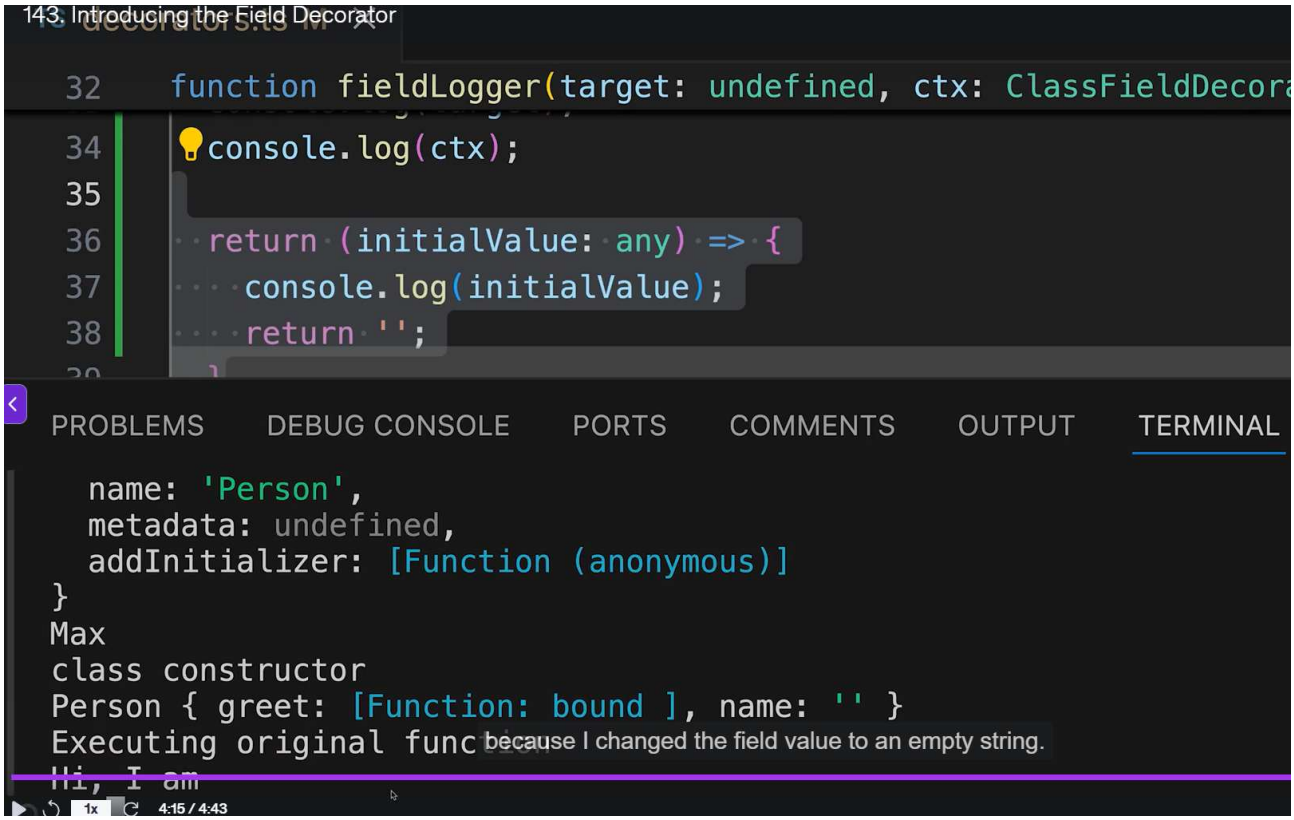
So to change the field or to be precise, it's value.

143. Introducing the Field Decorator

```
31
32  function fieldLogger(target: undefined, ctx: ClassFieldDecoratorContext)
33    console.log(target);
34    console.log(ctx);
35
36    return (initialValue: any) => {
37      console.log(initialValue);
38      return '';
39    }
40  }
41
42  @logger
43  class Person {
```

143. Introducing the Field Decorator

```
32  function fieldLogger(target: undefined, ctx: ClassFieldDecora
34    console.log(ctx);
35
36    return (initialValue: any) => {
37      console.log(initialValue);
```



Building Configurable Decorators with Factories

And a decorator factory in the end is just a function
that produces a decorator.
And why would you do that?
Well, let's say here for my field logger decorator,
where I'm changing the default value,
I actually want to be able to set the default value,